



UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Fundamentos de Programación



Cod. Asignatura: TAO207

Modelo: 1

Taller 3

Normas de Entrega:

- El nombre de cada archivo **.c**, archivo de entrada y archivo de salida debe coincidir con lo indicado por cada ejercicio.
- Debe colocar la siguiente información en la cabecera de cada archivo fuente por medio de un comentario de bloque: Fecha, Nombre, Apellido, Cédula, Número Sección y Número del Modelo del Parcial.
- La entrega de los programas se deberá realizar por medio de un archivo comprimido (**.zip**), el cual, debe contener una sub-directorio por cada ejercicio. Cada sub-carpeta deberá tener el archivo fuente (**.c**) y la entrada del programa correspondiente.
- No adjunte los archivos de salida del programa, ni adjunte los ejecutables generados por el compilador.
- Puede utilizar las librerías del estándar de C.
- Recuerde que solamente puede utilizar elementos del lenguaje C vistos en clase, tales como: condicionales, ciclos, archivos, acciones nominadas, arreglos, estructuras y matrices.
- Procure revisar el documento en su totalidad. El mismo contiene de 2 ejercicios.

No seguir las indicaciones antes mencionadas repercutirá en la nota final de su entrega.

Ejercicio 1. Nearest Neighbor Algorithm (10 ptos)

Los archivos PPM (con extensión **.ppm**) son una forma concisa y legible de representar imágenes en el computador (<https://netpbm.sourceforge.net/doc/ppm.html>), los cuales se pueden codificar en texto plano mediante la especificación P3, y en formato binario, por medio de la variante P6. En particular, los archivos P3 siguen el formato a continuación:

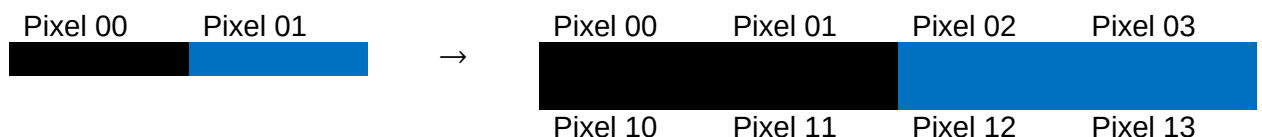
P3		<Nueva Línea>			
Anchura		Altura		<Nueva Línea>	
Profundidad de Color		<Nueva Línea>			
Pixel_11	Pixel_12	Pixel_13	...	Pixel_1M	<Nueva Línea>
...	...	...	...	...	<Nueva Línea>
Pixel_N1	Pixel_N2	Pixel_N3	...	Pixel_NM	<Nueva Línea>

Donde cada Pixel está constituido por los canales: Rojo, Verde y Azul.

Rojo (R)	Verde (G)	Azul (B)
----------	-----------	----------

Su trabajo como desarrollador consistirá en asistir al equipo artístico de la empresa de videojuegos en la que trabaja, de modo tal que cree un programa capaz de escalar o aumentar el tamaño de las imágenes con extensión **.ppm** que le estarán proporcionando continuamente.

Todas las imágenes que recibirá usará el formato PPM tipo P3, y debido a que dichas imágenes conforman parte del medio artístico Pixel Art (Arte de Píxeles), deberá utilizar el algoritmo Nearest Neighbor para preservar la nitidez de la imagen. Este algoritmo consiste en tomar un pixel P de la imagen original y repetirlo en el eje de la altura y anchura tantas veces lo indique un factor de escalado **E**. Por ejemplo, si $E = 2$:



Su programa deberá leer el factor de escalado **E** desde un archivo de configuración llamado: **escalado.conf** (recuerde que su programa se probará con distintos casos de prueba).

Ejemplo de Configuración: escalado.conf
32

Del mismo modo, su programa deberá **leer** la imagen desde la **entrada estándar** y **mostrar** la imagen **.ppm** resultante en la **salida estándar**.

Ello implica que para ejecutar su programa y guardar los resultados, deberá usar los operadores de redirección que ofrece bash:

./escalado < personaje.ppm > personaje-modificado.ppm

Un ejemplo de personaje lo podrá ver en la siguiente página.

De igual manera, **PROCURE LEER las propiedades adicionales** que debe cumplir su programa, las cuales están después de la página del ejemplo **personaje.ppm**.

Ejemplo de entrada estándar: **personaje.ppm**

P3
16 16
255
255
255
255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 201 11 0 201
11 0 201 11 0 165 17 41 165 17 41 255 255 255 255 255 255 255 255 255 255 255 255
255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 201 11 0 221 103 85 221
103 85 221 103 85 221 103 85 201 11 0 165 17 41 255 255 255 255 255 255 255 255 255
255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 129 0 56 221 103 85 201 11 0 201 11 0
201 11 0 201 11 0 201 11 0 201 11 0 129 0 56 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 201 11 0 201 11 0 201 11 0 78 125 195
110 156 204 110 156 204 110 156 204 110 156 204 78 125 195 255 255 255 255 255 255
255 255 255
255 255 255 255 255 255 255 255 255 255 221 103 85 129 0 56 201 11 0 78 125 195 147 201
227 147 201 227 147 201 227 255 255 255 255 255 255 255 147 201 227 110 156 204 255 255
255 255 255 255
255 255 255 255 255 255 221 103 85 201 11 0 129 0 56 201 11 0 58 100 163 147 201 227
147 201 227 147 201 227 147 201 227 147 201 227 147 201 227 110 156 204 255 255 255
255 255 255
255 255 255 255 255 255 129 0 56 201 11 0 97 12 49 201 11 0 165 17 41 58 100 163 110
156 204 110 156 204 110 156 204 110 156 204 58 100 163 255 255 255 255 255 255 255
255 255
255 255 255 255 255 255 97 12 49 201 11 0 78 6 24 201 11 0 201 11 0 165 17 41 165 17 41
165 17 41 165 17 41 165 17 41 129 0 56 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 78 6 24 129 0 56 78 6 24 165 17 41 201 11 0 201 11 0 201 11 0
201 11 0 201 11 0 201 11 0 78 6 24 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 78 6 24 97 12 49 78 6 24 129 0 56 201 11 0 201 11 0 201 11 0 201
11 0 201 11 0 201 11 0 78 6 24 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 78 6 24 78 6 24 129 0 56 165 17 41 201 11 0 201 11
0 201 11 0 201 11 0 201 11 0 78 6 24 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 78 6 24 129 0 56 129 0 56 97 12 49 97
12 49 129 0 56 129 0 56 165 17 41 78 6 24 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 78 6 24 129 0 56 129 0 56 97 12 49 255
255 255 255 255 255 78 6 24 129 0 56 78 6 24 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 175 198 205 175 198 205 78 6 24 78 6 24 78 6 24 78 6 24 175 198
205 175 198 205 78 6 24 78 6 24 78 6 24 177 192 197 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 175 198 205 175 198 205 175 198 205 175 198 205
175 198 205 175 198 205 175 198 205 175 198 205 175 198 205 175 198 205 255 255 255
255 255 255 255 255 255

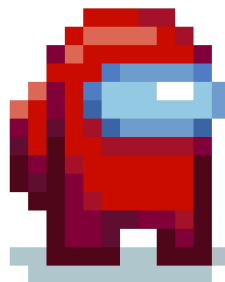


Figura 1. Representación gráfica del archivo **personaje.ppm**

Adicionalmente, su programa debe cumplir con las siguientes propiedades:

- $0 \leq \text{Profundidad de Color} \leq 255$.
- $0 \leq \text{Rojo, Verde, Azul} \leq \text{Profundidad de Color}$.
- $0 \leq \text{Altura, Anchura}$.
- La imagen generada por el programa no podrá tener unas dimensiones mayores a: 8192 x 8192 píxeles.
- Cada Pixel de la entrada estará separado por 1 espacio en blanco, 1 tabulador o 1 caracter nueva línea.
- Cada canal/componente del pixel estará separado por 1 espacio en blanco, 1 tabulador o 1 caracter nueva línea.
- $0 < E$, y además $E \in \mathbb{N}$
- Si $E = 1$, La imagen generada debe ser idéntica a la imagen original.
- El archivo fuente del programa deberá llamarse: **01-escalado.c**.
- El archivo ejecutable deberá llamarse **escalado**.
- El programa no debe alterar los colores de los pixeles originales. Ello implica que los colores que aparecerán en la imagen escalada deben ser los mismos que aquellos que están en la imagen original.
- Puede comprobar si su programa ha realizado correctamente la tarea asignada al abrir el archivo generado con el visor de imágenes de su preferencia.
- Su programa **no deberá mostrar** ningún mensaje en la terminal que haga referencia a la solicitud de los datos.
- Del mismo modo, El programa debe limitarse a mostrar únicamente el contenido en formato **.ppm** en la salida estándar. Cualquier mensaje adicional mostrado en la salida estándar afectaría el formato del archivo y con ello, su nota.

Ejercicio 2. Plaga y Depredador (10 ptos)

Existe un cierto tipo de insecto (llamado **la plaga**) que diezma los naranjos, y a su vez existe otro tipo que ataca a la plaga, al cual llamaremos **depredador**. Comúnmente, donde habita la plaga hay presencia de depredadores, sin embargo, éstos son torpes en el ataque, de modo que solo lo hacen si hay plaga en un entorno bastante cercano.

La empresa de agroquímicos KLRS C.A. lo ha contratado para que haga la simulación del comportamiento de ambos tipos de insectos durante lapsos variables en un terreno que tiene un arreglo de árboles de naranja de **fil x col**.

El equipo de biólogos de la empresa ha estudiado el fenómeno durante un tiempo considerable y ha llegado a la siguiente conclusión:

- 1) **Los depredadores eliminan plaga** (de uno en uno por ciclo de simulación) si están en los árboles cercanos o en donde se encuentran en cierto ciclo. Por cercano se tiene a los árboles vecinos en las ocho direcciones cardinales (Norte, Sur, etc.). Si no hay plaga en esos vecinos, debe trasladarse hacia cualquier dirección de manera aleatoria en busca de plaga.
- 2) **La plaga abandona un árbol**, solo si está sobre-poblado de plaga, y se dirige al árbol vecino, según la dirección del viento. Por sobre-población se entiende a la existencia de más de 10 ejemplares aproximadamente en un árbol, y la dirección del viento es hacia alguna dirección cardinal.
- 3) **El desplazamiento** de un árbol a otro y **la aniquilación de una plaga** por un **depredador** se hacen **por cada ciclo de simulación**, de uno en uno. La reproducción se hace por cada tres ciclos de simulación entre las parejas de cada especie, de uno en uno. Por ejemplo, si en un árbol hay 5 depredadores, implica la existencia de 2 parejas, lo que conlleva al nacimiento de 2 depredadores adicionales en el siguiente ciclo. Si en el árbol hay 1 depredador, significa que no hay parejas, por lo cual, el número de depredadores permanece igual.
- 4) Un depredador muere, cuando es minoría respecto a la plaga en un árbol **i** (se reduce el número de depredadores uno en uno por cada ciclo).

Por lo tanto, por cada árbol se tiene la siguiente información: el número de plaga y depredadores así como la dirección del viento, la cual puede variar aleatoriamente en cada ciclo de simulación hacia algunas de las direcciones cardinales.

Usted debe suministrarle esa información al programa de simulación que desarrolle, en un archivo llamado **sembradoin.txt**, el cual tiene la siguiente estructura:

```
cs
n m
fi ci npi ndi dvi
fj cj npj ndj dvj
.
.
.
fk ck npk ndk dvk
```

(Procure leer la página siguiente del enunciado)

Donde, $cs \geq 20$ es el número de ciclos de simulación, n y m son los números de fila y columna respectivamente del sembrado; $fi \leq n$ y $ci \leq m$, son las coordenadas del árbol; npi y ndi , son la cantidad de plaga y depredadores en el árbol i respectivamente, y dvi es la dirección inicial del viento (durante la simulación, ésta varía aleatoriamente) según la siguiente lista:

N = 0, S = 1, E = 2, O = 3, NE = 4, NO = 5, SE = 6, SO = 7.

El programa a desarrollar debe generar un archivo de salida llamado **sembradoout.txt**, que contenga “**pantallas**” de la configuración del sembrado, de acuerdo al siguiente formato:

Debe aparecer una **M**, si en cierto árbol hay tanto plaga como depredadores, una **P** si hay plaga solamente, y una **D** para depredadores solamente. Si en el árbol no hay ninguno de los dos tipos, se deja vacío.

El número de pantallas a aparecer son cinco: La configuración inicial del sembrado justo después de leer el archivo de entrada, la configuración cuando han transcurrido el 25%, 50%, 75%, 100% de los ciclos respectivamente.

Tome en cuenta que el programa deberá llamarse: **02-simulacion.c**

Dado que su programa deberá generar números aleatorios, considere utilizar la siguiente rutina en su código fuente para cumplir con dicho requerimiento:

```
#include<stdlib.h>
#include<time.h>

// aleatorio(0, 10) -> Genera números aleatorios entre 0 y 10
int aleatorio(int minimo, int maximo){
    static int inicializado = 0;
    if( !inicializado ){
        srand(time(NULL));
        inicializado = 1;
    }

    int valor = rand();
    int distancia = maximo - minimo + 1;
    int ajustado = minimo + valor / (RAND_MAX / distancia + 1);

    return ajustado;
}
```